

Capítulo 3: La capa de transporte

Los protocolos de capa de transporte se implementan en los **hosts**, no en los routers.

Convierten los mensajes a enviar en paquetes llamados **segmentos**. Y este segmento se lo pasa a la capa de red y lo transforma en un **datagrama**.

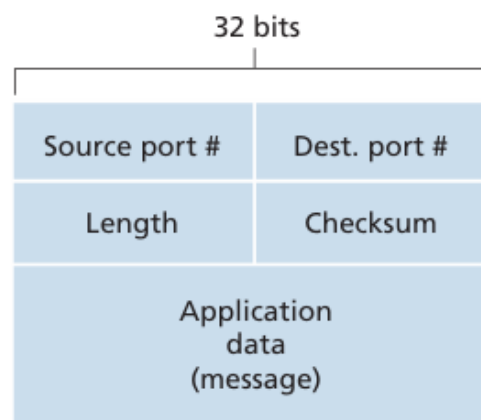
▼ Multiplexación y Demultiplexación

- **Demultiplexación:** tarea destinada a entregar los datos de un segmento de la capa de transporte al socket correspondiente. (Capa de Red a Capa de Transporte)
- **Multiplexación:** tarea de reunir los fragmentos de dicho socket para crear los segmentos y pasarlos a la capa de red. (Capa de Transporte a Capa de Red)

▼ UDP

Estructura

Provee la función de multiplexar/demultiplexar y algún mecanismo de comprobación de errores.



Esta es la estructura del segmento UDP. Requiere nada más del puerto origen y destino.

A su vez, cuenta con un checksum para validar que los datos enviados fueron correctos.

El checksum es el complemento a 1 de la suma de todas las palabras de 16 bits en el segmento.

El header UDP tiene un tamaño de 8 bytes.

▼ Construyendo un protocolo RDT

Para hacer esto posible se necesita que sean protocolos **ARQ**.

ARQ (Automatic Repeat reQuest): se basa en retransmitir mensajes cuando estos no fueron recibidos correctamente.

Por cada mensaje el receptor da el OK o pide que se le repita.

Requiere 3 capacidades: detección de errores, feedback del receptor y retransmisión.

Stop & Wait

El emisor envía **un solo paquete** y se queda esperando la confirmación (ACK) del receptor antes de enviar el siguiente.

Funcionamiento:

1. El emisor envía el paquete i .
2. El emisor activa el timer.
3. Si el ACK llega antes del timeout, el emisor envía el paquete $(i+1)$
4. Si hay timeout (porque se perdió el ACK o el paquete mismo), el emisor vuelve a enviar el paquete i .

Pipelining

La idea es que se permita enviar muchos paquetes sin tener que esperar que se reconozca cada uno. Y de acá salen dos approaches: **GBN** y **SR**.

Go Back N

Permite al emisor enviar múltiples paquetes sin esperar confirmación, hasta un límite máximo de N paquetes (el tamaño de la ventana de transmisión).

Funcionamiento:

1. El emisor puede enviar paquetes de forma continua mientras que no supere N.
2. El receptor solo acepta paquetes en orden. Si se recibe el 1 y el 3, pero el 2 se perdió, el receptor descarta el 3 (y cualquier otro posterior) y vuelve a enviar un ACK para el paquete 1.
3. Utiliza ACKs acumulativos, un ACK con el numero k indica que el receptor recibió correctamente todos los paquetes hasta el k.
4. El emisor mantiene un único timer para el paquete más antiguo que aún no ha sido confirmado. Si hay timeout, el emisor asume que ese paquete se perdió y vuelve a retransmitir todos los paquetes que envió a partir de ese punto.

Selective Repeat

El emisor retransmite únicamente aquellos paquetes de los que sospecha que se perdieron.

Funcionamiento:

1. Al igual que GBN, el emisor tiene una ventana de tamaño N y envía paquetes de manera continua.
2. El receptor acepta y almacena en un buffer los paquetes desordenados.
3. El receptor envía ACKs individuales para cada paquete específico que recibe correctamente, sin importar el orden.
4. El emisor mantiene un timer por cada paquete enviado. Si hay timeout, retransmite el paquete que expiró.
5. Una vez que el receptor recibe el paquete que faltaba, entrega todos los paquetes ordenados a la capa superior y desplaza su ventana de recepción.

▼ TCP

Como ya bien se mencionó en la capa de aplicación, TCP es:

- Orientado a conexión
- Full-dúplex
- Conexión punto a punto
- Usa un three way handshake para establecer la conexión

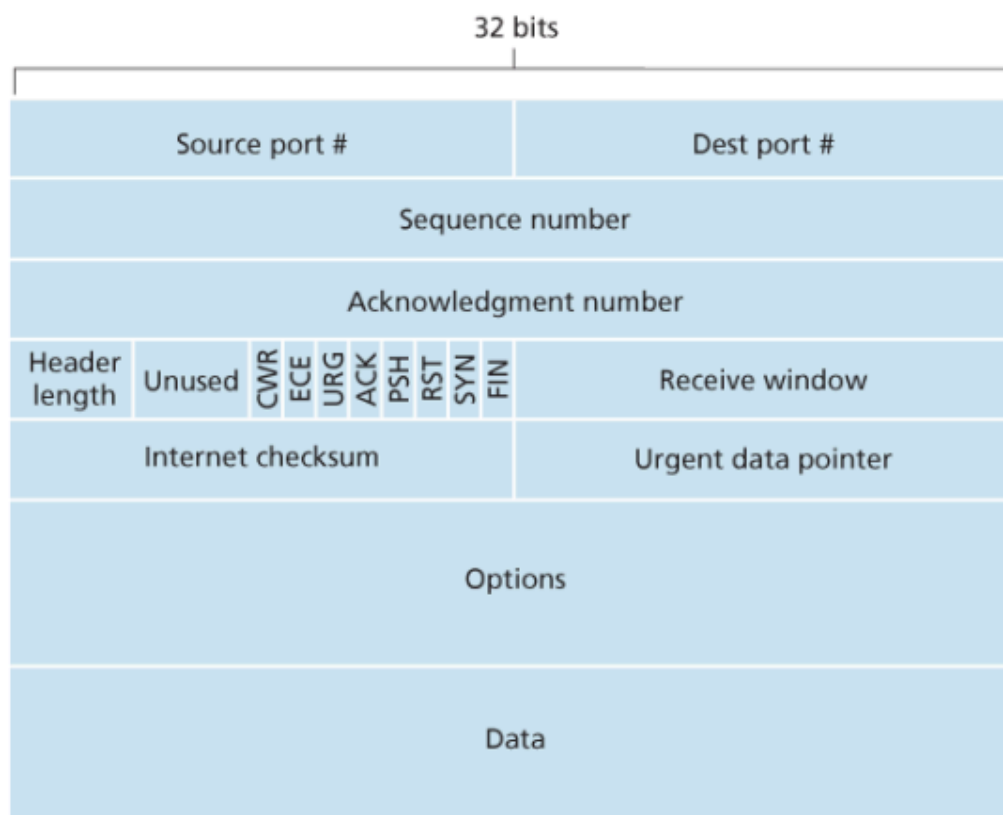
SYN → SYN/ ACK → ACK

- Y para cerrarla usa el bit de FIN y cierra en 4 pasos:

FIN → FIN/ACK → FIN → ACK

La cantidad máxima de datos que se pueden meter dentro de un segmento está limitada por el **MSS (Maximum Segment Size)**. Que no puede superar el **MTU (Maximum Transmission Unit)**, por lo tanto el MSS va a estar condicionado por eso más la suma de los headers de TCP/IP (generalmente 40 bytes).

Estructura



El header TCP tiene 20 bytes, al igual que UDP tiene puerto origen y destino para multiplexar y demultiplexar.

A su vez tiene campos interesantes:

- Sequence number (32 bits): sirve para implementar el rdt.
- Receive window (ventana de recepción, 16 bits): se usa para el control de flujo. Se emplea para indicar el número de bytes que un receptor está dispuesto a aceptar.

- Bits tales como SYN, RST, ACK y FIN.

Estimación del RTT y Timer

Se detallan 3 formas de estimar el RTT y una para el timeout:

- **SampleRTT**: es el tiempo que transcurre desde que un segmento se entrega a la capa de red hasta que llega su ACK.
- **EstimatedRTT**: calcula un promedio móvil ponderado llamado **EWMA (Exponential Weighted Moving Average)**.

$$EstimatedRTT = (1 - \alpha) \cdot EstimatedRTT + \alpha \cdot SampleRTT$$

El valor de α es 0,125

- **DevRTT**: si el RTT cambia mucho, hay que calcular la desviación:

$$DevRTT = (1 - \beta) \cdot DevRTT + \beta \cdot |SampleRTT - EstimatedRTT|$$

El valor de β es 0,25

- **TimeoutInterval**: es el promedio más un "margen de seguridad" de 4 veces la desviación:

$$TimeoutInterval = EstimatedRTT + 4 \cdot DevRTT$$

Transferencia de datos fiable

1. TCP utiliza **un único temporizador de retransmisión**. Y este timer lo tiene el segmento más antiguo que todavía no fue confirmado.
2. Se plantean 3 situaciones de robustez:
 - a. **ACK perdido**: El receptor recibe bien, pero su confirmación se pierde. El emisor retransmite por timeout. El receptor ve que el número de secuencia es duplicado y lo descarta, pero **vuelve a mandar el ACK**.
 - b. **Timeout prematuro**: Si el emisor retransmite el primer segmento, pero antes de que expire el segundo llega su ACK, TCP es lo suficientemente inteligente para no retransmitir el segundo innecesariamente.
 - c. **ACK acumulativo al rescate**: Si se pierden ACKs intermedios, pero llega un ACK posterior, el emisor sabe que todo llegó bien y **no retransmite nada**.

3. Se aplican 2 técnicas para mejorar el rendimiento:

a. **Duplicación del intervalo (Backoff Exponencial):**

Cada vez que ocurre un timeout, se duplica el tiempo de espera para la siguiente retransmisión. Esto es para no retransmitir agresivamente.

b. **Retransmisión rápida (Fast Retransmit):**

Esperar a un timeout puede ser lento, entonces se utiliza los **ACKs duplicados**:

Si el emisor recibe **3 ACKs duplicados** para el mismo dato, asume que el siguiente segmento se perdió y lo retransmite de inmediato, sin esperar el timeout.

4. TCP es un protocolo híbrido entre GBN y SR:

- Se parece a GBN, porque usa ACKs acumulativos y mantiene solo la base y el siguiente número de secuencia.
- Se parece a SR, porque algunas implementaciones guardan en un buffer los datos desordenados y gracias a **SACK** (ACKs Selectivos) pueden confirmar bloques de datos específicos.

Control de flujo

Es el mecanismo que utiliza TCP para evitar que un emisor "ahogue" rápido a un receptor lento. Todos los datos que se envía se quedan en un buffer de recepción hasta que la app tenga tiempo de procesarlos.

El control de flujo es el "freno de mano" que el receptor activa para avisarle al emisor que baje la velocidad.

Para gestionar este espacio, el receptor monitorea tres variables:

- **RcvBuffer**: tamaño total de la memoria asignada a esa conexión.
- **LastByteRead**: puntero que indica hasta donde leyó la app.
- **LastByteRcvd**: puntero que indica qué datos ya llegaron de la red y están guardados.

La **ventada de recepción (rwnd)** es el espacio libre real que queda:

$$rwnd = RcvBuffer - (LastByteRcvd - LastByteRead)$$

Con esto el receptor incluye su valor actual de **rwnd** en el campo de **receive window** del header TCP de cada **ACK** que envía de vuelta.

Como restricción, el emisor lleva la cuenta de cuánto ha enviado, pero sin confirmación y debe cumplir la siguiente regla:

$$LastByteSent - LastByteAcked \leq rwnd$$

Esto garantiza que el emisor nunca envíe más de lo que el receptor puede guardar en ese momento.

¿Qué pasa si **rwnd = 0**?

Si el receptor anuncia **rwnd = 0**, el emisor continúa enviando segmentos de **1 byte de datos** de forma periódica. El receptor está obligado a responder estos pequeños segmentos con un ACK que contenga el nuevo valor de la ventana. Cuando la ventana sea mayor a cero, la comunicación se reanuda normalmente.

Control de congestión

Mientras que el control de flujo protegía al receptor, este protege a la red entera. El emisor debe deducirlo solo. Sirve para regular la cantidad de datos que se envía por la red.

TCP añade una variable llamada **cwnd** (Ventana de congestión).

El emisor debe respetar dos límites a la vez:

$$LastByteSent - LastByteAcked \leq \min(cwnd, rwnd)$$

Una forma de probar el ancho de banda con esto sería así:

- Si llega un ACK, el emisor aumenta el tamaño de **cwnd**.
- Si ocurre un timeout o 3 ACKs duplicados, el emisor reduce el tamaño de **cwnd**.

También si los ACKs llegan rápido (red libre), se aumenta la **cwnd** rápido. Pero si tardan en llegar, aumenta lentamente.

Slow Start (Reno)

Es la fase inicial de una conexión TCP y en verdad el nombre engaña, crece de forma exponencial. Su objetivo es descubrir rápidamente cuánto ancho de

banda hay disponible en la red sin saturarla de golpe.

- **Estado inicial:** El valor de **cwnd** se inicia en **1 MSS**.
- **Por cada ACK recibido**, aumenta en **1 MSS** la **cwnd**.
- El efecto acumulado es que, como los ACKs son acumulativos, aproximadamente cada **RTT** se duplica la **cwnd**.

N segmentos enviados → N ACK recibidos → **cwnd = cwnd + N = 2 * cwnd (1 RTT)**

¿Cuándo termina el slow start?

Existen 3 posibles casos:

A. Timeout:

1. **Reset:** El **cwnd** vuelve a valer **1 MSS**.
2. **Umbral:** Se actualiza el valor del **ssthresh** a **cwnd/2**
3. **Reinicio:** Arranca de nuevo el slow start desde 1.

B. Alcanzar el umbral (ssthresh)

Pasa al estado de **Congestion Avoidance**, donde deja de ser exponencial y pasa a ser lineal.

C. 3 Acks duplicados

Hace un **Fast Retransmit** y entra en el estado de **Fast Recovery**.

Congestion Avoidance (Reno)

En este punto TCP asume que la última vez que hubo problemas fue al doble de esa velocidad.

Suma de a **1 MSS por cada ventana entera ACKeada**. El efecto acumulado es que, como los ACKs son acumulativos, aproximadamente por cada RTT aumente la cwnd en 1 MSS.

La regla clásica para calcular la cwnd por cada ACK nuevo recibido es:

$$cwnd = cwnd + \frac{(1MSS)^2}{cwnd}$$

Si durante esta etapa hubo un **timeout** pasa:

1. El **ssthresh** pasa a valer **cwnd/2**.
2. La **cwnd** pasa a valer **1 MSS**.

3. Volvemos al **Slow Start**.

Pero si ocurren 3 ACKs duplicados, se hace **Fast Retransmit** y se transiciona a **Slow Start** haciendo lo mismo que en un **timeout** (en Tahoe), o **Fast Recovery** (en Reno).

Fast Recovery (Reno)

Antes de entrar a este estado siempre va **Fast Retransmit**. Al entrar en este estado:

1. El **ssthresh** pasa a valer **cwnd/2**.
2. La **cwnd** se reduce a la mitad.

Permite que una conexión TCP sea resiliente y no pierda toda su velocidad ante una pérdida de paquetes puntual.

- **Inflado de la ventana:** Por cada **ACK duplicado adicional** que llega mientras espera el arreglo, incrementa la **cwnd en 1 MSS**. Esto se hace porque cada ACK duplicado indica que un segmento salió de la red y llegó al receptor, liberando espacio.
- **Finalización:** Una vez que llega el ACK del segmento que faltaba, TCP desinfla **cwnd** al valor del **ssthresh** y vuelve al estado de **Congestion Avoidance**.
- **Excepción:** Si ocurre un **timeout** durante esta fase, **cwnd** pasa a valer **1 MSS** y vuelve a **Slow Start**.

TCP Tahoe vs TCP Reno

Dependiendo de cual usemos cambia la forma en la que se reaccionamos a los **3 ACKs duplicados**:

El **ssthresh** es **cwnd/2** y la **cwnd**:

Característica	Tahoe (Antiguo)	Reno (Moderno)
Timeout	Baja a 1 MSS	Baja a 1 MSS
3 ACKs duplicados	Baja a 1 MSS	Baja a la mitad
Siguiente estado	Slow Start	Fast Recovery